UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA

Universidad Técnica Federico Santa María

Departamento de Informática
Universidad Técnica Federico Santa María

Visualización de Datos

Tarea 1

Benjamín Romero Pérez ROL: 202273060-9
Benjamin Ferrada Larach ROL: 202273061-7

Índice

1. Trabajo Individual **2**
1.1. Criterios seleccionados: 2
1.2. Justificación elecciones: 2
1.3. Gráficos obtenidos: 3
1.4. Conclusiones: 4

2. Trabajo Grupal **5**
2.1. Prompt a la IA 5

Índice de figuras

1. Gráfico Criterios Benjamín Romero: Obtenidos de https://assets.deadlock-api.com/ . . 3
2. Gráfico Criterios Benjamín Ferrada: Obtenidos de https://assets.deadlock-api.com/ . . 3
3. Gráfico entregado por Gemini Pro 3.1: Obtenidos de https://assets.deadlock-api.com/ . 8

1. Trabajo Individual

1.1. Criterios seleccionados:

- Benjamín Romero:
  - Popularidad del ítem vs cantidad de atributos que otorga.
  - Costo total de build Endgame vs build Mínima por héroe.
- Benjamín Ferrada:
  - Distribución de costos de las habilidades (Tier 1 a 4) por categoría de item (Weapon, Vitality, Spirit)
  - Densidad de atributos por cada alma invertida.

1.2. Justificación elecciones:

- Benjamín Romero:
  Elegí estos criterios por las siguientes razones:
  - Popularidad del ítem vs cantidad de atributos que otorga: Este criterio busca determinar si existe una relación entre la popularidad de un ítem entre los héroes y la cantidad de atributos que este otorga. Permite analizar si los jugadores y el diseño del juego tienden a favorecer ítems con mayor cantidad de atributos, o si la popularidad responde a otros factores como el tipo de efecto o la categoría del ítem.
  - Este criterio analiza la inversión económica necesaria para construir la build óptima de cada héroe, comparando dos escenarios: la build endgame (los 12 ítems Good más caros) y la build mínima viable (los 12 ítems Good más baratos) el sistema de almas de Deadlock es la economía que cambia en cada partida, y entender cuánto cuesta buildear a cada héroe permite identificar cuáles son más accesibles de construir en etapas tempranas y cuáles requieren mayor inversión para alcanzar su power spike.
- Benjamín Ferrada:
  La razón por las que elegí estos criterios son las siguientes:
  - Distribución de costos por categoría (Criterio 1): Este criterio permite visualizar la estructura de progresión de Deadlock. Al segmentar los items por Tier y Categoría (Weapon, Vitality, Spirit), podemos identificar qué ramas de especialización requieren una mayor acumulación de .ªlmas"para alcanzar su máximo potencial en las distintas fases de la partida.
  - Densidad de atributos por alma invertida (Criterio 2): Se eligió para medir la eficiencia técnica de la inversión económica. Este criterio responde a la pregunta de si los items de alto costo (Tier 3 y 4) mantienen la misma rentabilidad estadística que los items básicos, o si su valor reside principalmente en efectos únicos no cuantitativos.

1.3. Gráficos obtenidos:

- Benjamín Romero:

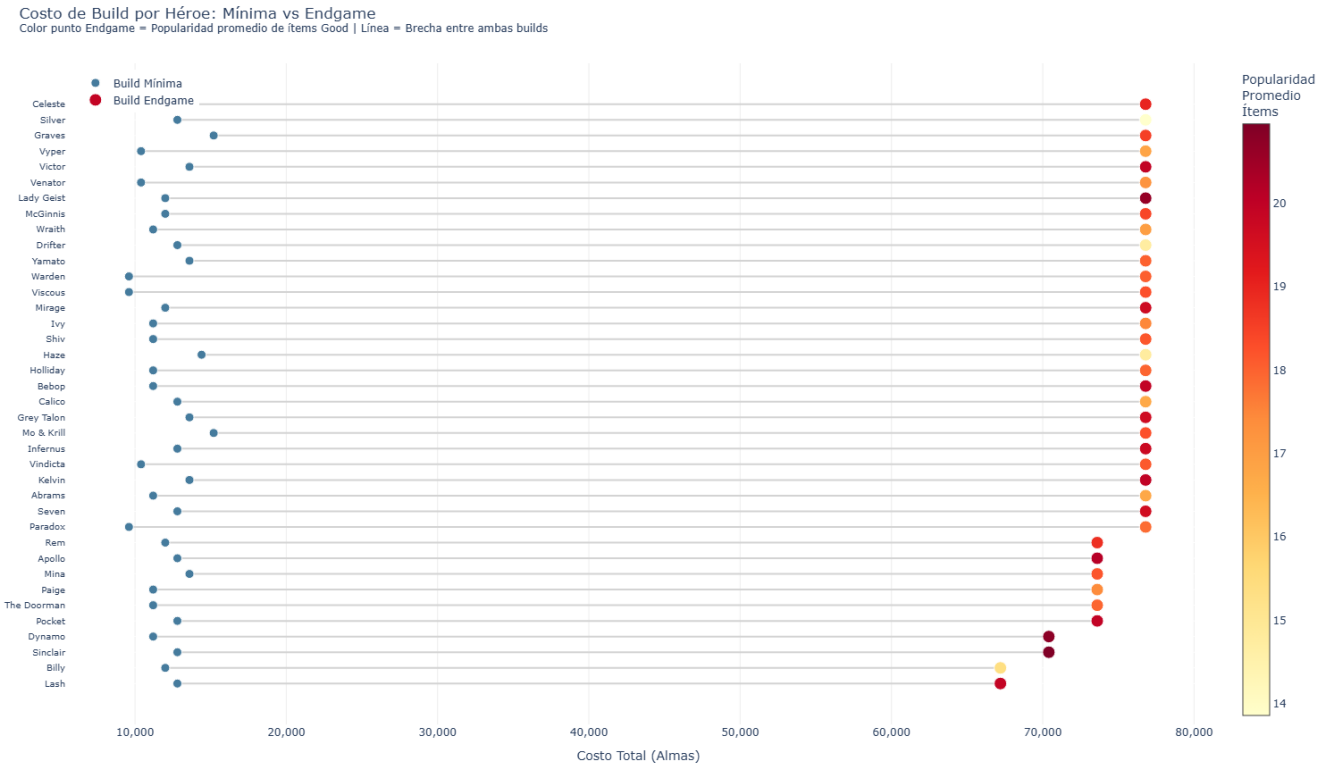

Figura 1: Gráfico Criterios Benjamín Romero: Obtenidos de https://assets.deadlock-api.com/

- Benjamín Ferrada:

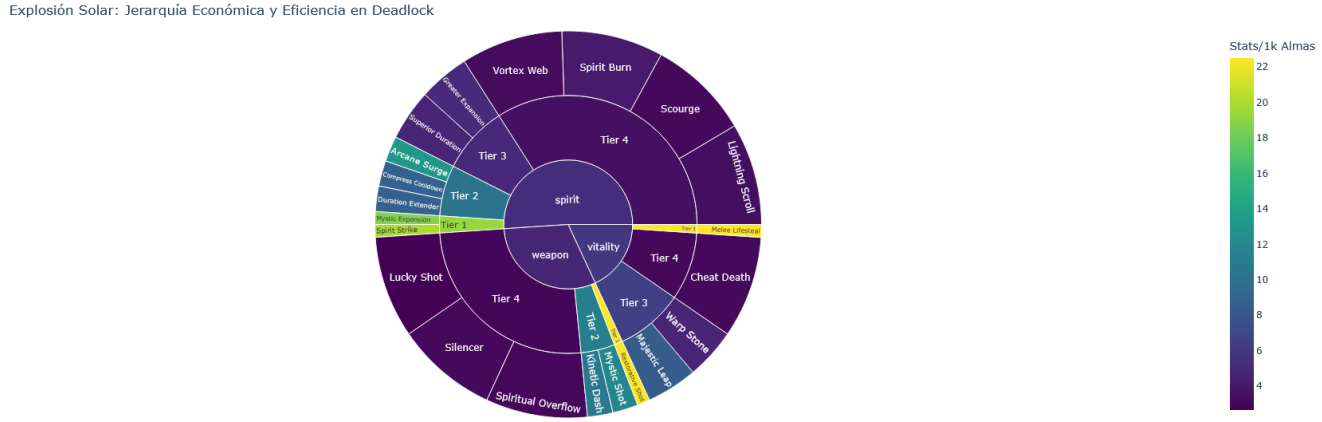

Figura 2: Gráfico Criterios Benjamín Ferrada: Obtenidos de https://assets.deadlock-api.com/

1.4. Conclusiones:

- Benjamín Romero:
  La brecha Endgame vs Mínima es consistente entre casi todos los héroes La mayoría de los héroes muestra una diferencia muy similar entre su build mínima y endgame, con los puntos azules concentrados entre 10.000 y 15.000 almas y los rojos agrupados alrededor de 75.000-77.000. Esto sugiere que el juego está diseñado con una economía bastante equilibrada entre personajes.

  La escala de color de los puntos endgame se mantiene en tonos naranjos/rojos para casi todos los héroes (entre 14 y 20 de popularidad promedio), con pocas excepciones como Dynamo y Sinclair que muestran colores más oscuros, indicando que sus ítems Good son más populares entre el roster general.

- Benjamín Ferrada:
  Tras analizar la visualización generada (el Sunburst Chart), se establecen las siguientes conclusiones:

  Eficiencia decreciente en el escalado: Los items de Tier 1 presentan la densidad de atributos más alta del gráfico. Esto demuestra que el diseño del juego incentiva una fase inicial de compras rápidas para obtener picos de poder inmediatos, sacrificando eficiencia económica a medida que el jugador avanza hacia items de Tier 4.

  Especialización de la rama Spirit: Se observa que la categoría Spirit concentra una mayor proporción de su valor total en items de Tier 3 en comparación con la categoría Weapon. Esto sugiere que los héroes basados en habilidades dependen de una economía de "juego medio"(mid-game) mucho más robusta para ser competitivos.

  Balance de Vitalidad: La categoría Vitality muestra una distribución de densidad más uniforme entre Tiers, lo que indica que la supervivencia escala de forma más constante y predecible que el daño o la utilidad de habilidades a lo largo de la partida.

2. Trabajo Grupal

2.1. Prompt a la IA

IA seleccionada: Gemini Pro 3.1

Prompt realizado:
Ayúdame a crear una visualización de datos para un videojuego nuevo de Valve llamado Deadlock.

Ya tengo estos análisis: Popularidad del ítem vs atributos Costo de builds vs build mínima por héroe Costos de habilidades por tier y tipo de ítem Atributos por alma invertida
Quiero una nueva visualización que conecte todo eso bajo una misma idea: qué tan eficiente es gastar recursos (almas) en ítems y builds.

Necesito que: Propongas una visualización clara (y que no repita lo anterior) Expliques brevemente por qué sirve Me des el código en Python (matplotlib, seaborn o plotly) Me digas qué variables usar Me ayudes a unir los datasets Opcionalmente agregale la foto a los personajes

Ojalá sea algo fácil de entender y útil para tomar decisiones en el juego.
Adicionalmente, se le entregó la celda de Jupyter con los Dataset

Código entregado por la IA:

```
import pandas as pd
import plotly.express as px

def generar_ternario_definitivo_con_imagenes(df_crit1, df_crit2, df_benyen, df_heroes):
    """
    Genera un Grfico Ternario unificando la composicin econmica de la build (Romero)
    con la eficiencia tcnica de los tems (Ferrada).
     Incluye las imgenes de los hroes en el hover!
    """
    print("Calculando el balance econmico del Meta y cargando imgenes...")

    # 1. Filtrar solo las builds Endgame de Romero
    df_endgame = df_crit2[df_crit2['tipo_build'] == 'Endgame'].copy()

    # 2. Calcular los porcentajes de inversin para el tringulo
    df_endgame['pct_weapon'] = (df_endgame['costo_weapon'] / df_endgame['costo_total']) *
        100
    df_endgame['pct_vitality'] = (df_endgame['costo_vitality'] /
        df_endgame['costo_total']) * 100
    df_endgame['pct_spirit'] = (df_endgame['costo_spirit'] / df_endgame['costo_total']) *
        100

    # 3. Calcular mtricas combinadas (Popularidad y Densidad)
    filas_combinadas = []
    for _, row in df_endgame.iterrows():
        heroe_name = row['heroe']
        items_build = row['items_good'] # Lista de class_names

        # Obtenemos el class_name del hroe (para la imagen)
        # Buscamos en df_heroes el class_name que corresponde a este heroe_name
        match_heroe = df_heroes[df_heroes['name'] == heroe_name]
        clase_heroe = match_heroe.iloc[0]['class_name'] if not match_heroe.empty else ""

        # Datos Ferrada (Densidad)
        df_f = df_benyen[df_benyen['class_name'].isin(items_build)]
        avg_densidad = df_f['densidad_stats'].mean() if not df_f.empty else 0

        # Datos Romero (Popularidad)
        df_r = df_crit1[df_crit1['class_name'].isin(items_build)]
        avg_pop = df_r['popularidad_good'].mean() if not df_r.empty else 1

        filas_combinadas.append({
            'heroe': heroe_name,
            'clase_heroe': clase_heroe, # Guardamos el ID interno para la foto
            'avg_densidad': avg_densidad,
            'avg_popularidad': avg_pop
```

```
    })

df_metricas = pd.DataFrame(filas_combinadas)
df_final = pd.merge(df_endgame, df_metricas, on='heroe')

# ---> AQU AGREGAMOS LA MAGIA DE LA IMAGEN <---
url_base = "https://assets.deadlock-api.com/images/heroes/"
# Creamos la etiqueta HTML de la imagen usando el class_name del hroe
df_final['img_hover'] = df_final['clase_heroe'].apply(
    lambda clase: f"<img src='{url_base}{clase}_icon.webp' width='60' height='60'>" if
        clase else ""
)

# 4. GENERACIN DEL GRFICO TERNARIO
fig = px.scatter_ternary(
    df_final,
    a="pct_weapon",   # Eje Superior
    b="pct_vitality", # Eje Inferior Izquierdo
    c="pct_spirit",   # Eje Inferior Derecho
    hover_name="heroe",
    size="avg_popularidad",
    color="avg_densidad",
    color_continuous_scale="Turbo",
    title="Balance del Meta: Distribucin Econmica y Eficiencia por Hroe",
    labels={
        "pct_weapon": "% Inversin en Weapon",
        "pct_vitality": "% Inversin en Vitality",
        "pct_spirit": "% Inversin en Spirit",
        "avg_densidad": "Eficiencia<br>(Stats/1k Almas)"
    },
    size_max=30,
    # Pasamos la imagen oculta en los datos del hover
    hover_data={"img_hover": False, "clase_heroe": False, "items_good": False}
)

# ---> CONFIGURAMOS EL HOVER PARA QUE MUESTRE LA IMAGEN <---
fig.update_traces(
    # customdata[0] ser nuestra columna 'img_hover'
    hovertemplate=(
        "%{customdata[0]}<br><br>" # Aqu renderiza la foto del hroe
        "<b>%{hovertext}</b><br><br>" # Nombre del hroe
        "Weapon: %{a:.1f}%<br>"
        "Vitality: %{b:.1f}%<br>"
        "Spirit: %{c:.1f}%<br>"
        "Eficiencia: %{marker.color:.1f} stats/1k almas<br>"
        "<extra></extra>" # Quita el cuadro lateral
    )
)
```

```
    # Mejoras visuales del tringulo
    fig.update_layout(
        ternary=dict(
            sum=100,
            aaxis=dict(title="Weapon (%)", ticksuffix="%", min=0.01, linewidth=2),
            baxis=dict(title="Vitality (%)", ticksuffix="%", min=0.01, linewidth=2),
            caxis=dict(title="Spirit (%)", ticksuffix="%", min=0.01, linewidth=2),
            bgcolor="rgba(240, 240, 240, 0.8)"
        ),
        paper_bgcolor="white",
        font=dict(size=12)
    )

    fig.show()
    return df_final

# ==========================================
#  CMO  EJECUTARLO EN TU CELDA FINAL:
# ==========================================
# (Asegrate  de haber ejecutado antes df_crit1, df_crit2, df_benyen y de tener
    df_heroes_romero cargado)
df_mega_ternario = generar_ternario_definitivo_con_imagenes(df_crit1, df_crit2,
    df_benyen, df_heroes_romero)
```

Visualización Creada:

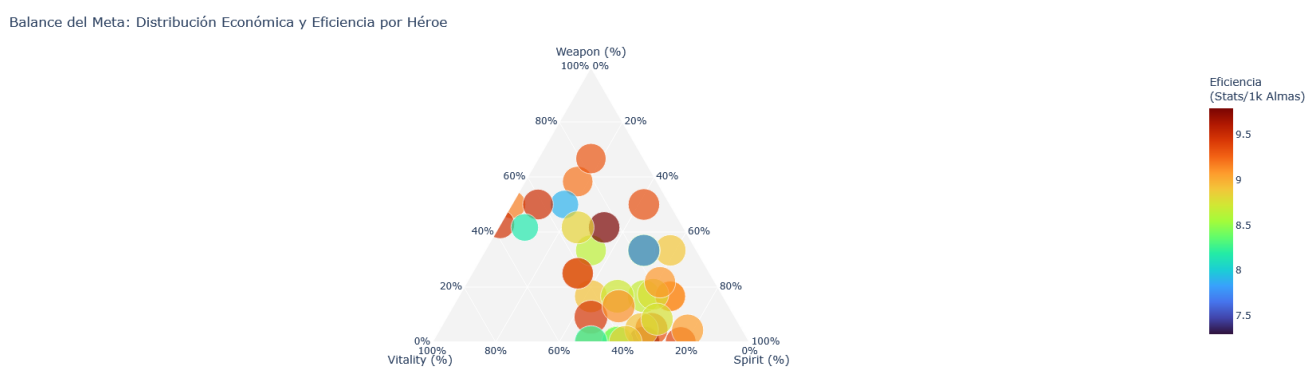

Figura 3: Gráfico entregado por Gemini Pro 3.1: Obtenidos de https://assets.deadlock-api.com/